

Document Intelligence Asynchronous Pipeline

Upload any document, get a job id back in milliseconds, and receive its text when processing finishes. Uses the claim-check pattern, survives crashes and runs fully on-device.

Deliverable: source code (runs with `docker compose up --build`) and this report. The benchmark, chaos test results and every number in this report can be reproduced with the scripts in the repository.

1. Product overview	6. Part 4 · Queue design
2. Architecture	7. Part 5 · Failure handling (chaos tests)
3. Part 1 · Ingestion & storage	8. Part 6 · Monitoring & UI
4. Part 2 · Job tracking	9. Trade-offs, limits & future work
5. Part 3 · Document intelligence (benchmark)	10. Appendix · How to run & reproduce

1. Product overview

Problem. Turning documents into machine-readable text is slow and unpredictable. A text file takes milliseconds, while a 30-page scan needs about a minute of OCR. Files also arrive in bursts, and many are malformed. A synchronous endpoint would hold the connection open, time out and lose work when a process dies.

Product. *DocIntel* is a self-hosted service with an HTTP API, a web UI and a monitoring dashboard. Clients push a document and get a job id back right away. They can then poll the job (or register a webhook) to follow its progress, and download the extracted text together with per-page details and metadata.

For API consumers

- `POST /v1/documents` → 202 + job id
- status, stage, progress %, queue position, warnings
- text, per-page JSON, entities, webhook

Formats (by content, not extension)

- PDF (digital, scanned or mixed)
- PNG, JPG, TIFF, WebP, BMP, GIF
- TXT/MD/CSV/JSON/XML (any encoding), HTML, DOCX, PPTX, XLSX, RTF

Operational guarantees

- no job lost on worker, broker or full-stack crash
- every job ends SUCCEEDED / FAILED / CANCELLED
- 100% on-device, open-source components

Value-adding payload parameters. These were chosen because each one changes cost or behaviour in a way only the client knows about:

Parameter	Default	Why it exists
<code>language</code>	<code>eng+spa</code>	OCR accuracy depends on the language model. The default suits a bilingual (Mexican) user base. The value is validated against the installed traineddata, so a typo returns 422 instead of a failed job later.
<code>ocr_mode</code>	<code>auto</code>	<code>auto</code> runs OCR only on pages without a usable text layer. <code>force</code> re-OCRs PDFs whose text layer is garbage (a common scanner artefact). <code>off</code> saves minutes when only the embedded text matters.
<code>ocr_engine</code>	<code>rapidocr</code>	<code>rapidocr</code> is the most accurate on noisy scans and photos. <code>tesseract</code> is about 3× faster and uses about 4× less memory, and suits clean bulk scans (benchmark in §5).
<code>force</code>	<code>false</code>	Uploads are content-addressed (SHA-256), so identical bytes with identical options return the stored result instantly (HTTP 200, <code>cached_from_job_id</code>). <code>force=true</code> bypasses this.

<code>extract_entities</code>	<code>true</code>	Emails, URLs, dates, amounts and phone numbers are cheap regex metadata that help triage invoices or forms.
<code>client_reference</code>	-	Correlates jobs with the caller's own records (filterable).
<code>callback_url</code>	-	Push instead of poll: a webhook fires on terminal state, retried with exponential backoff.

2. Architecture

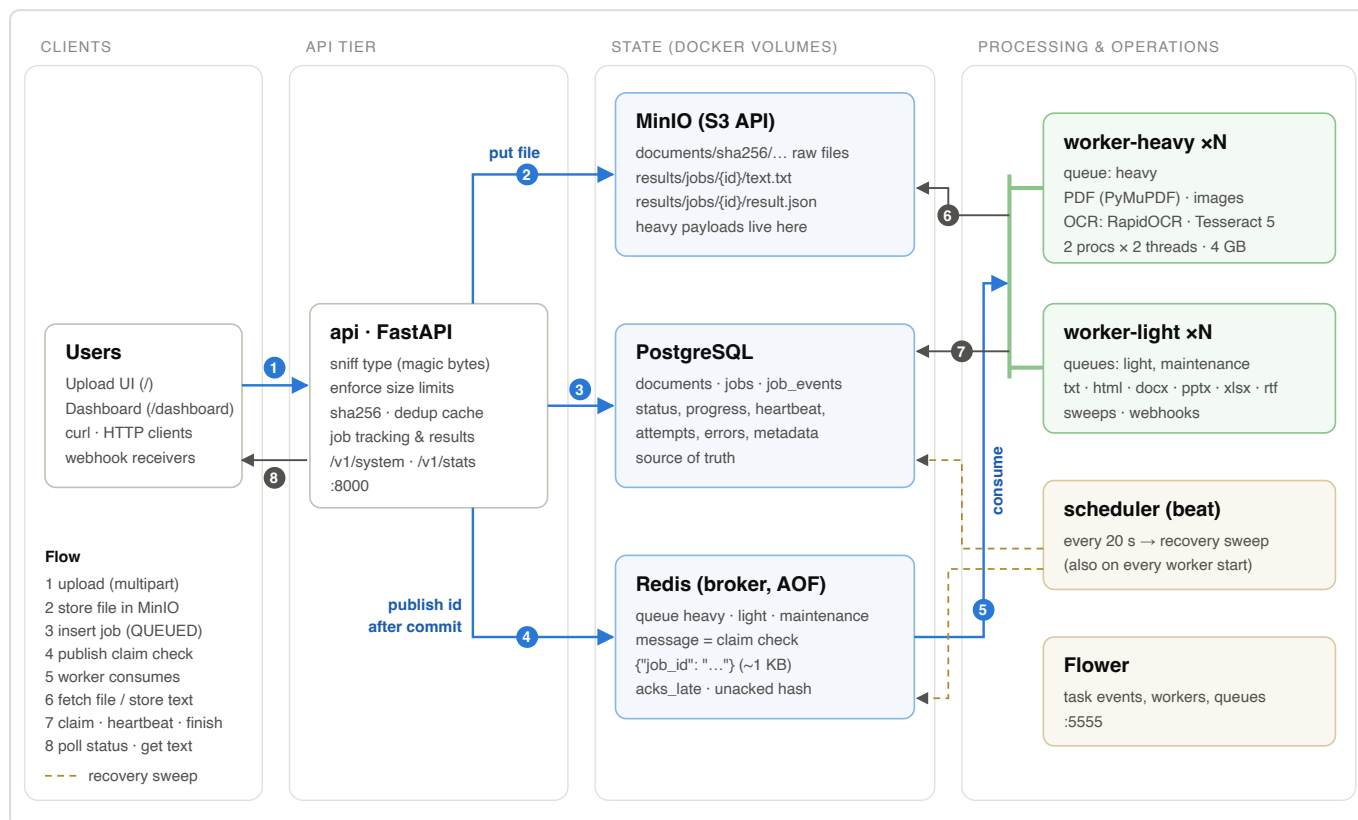


Figure 1. Components and the claim-check flow. Heavy bytes move only between the API, MinIO and the workers. The broker carries only a job id, and PostgreSQL is the single source of truth for job state. Everything runs as containers from one compose file, and the three stateful services keep their data in named volumes.

Service	Technology	Responsibility	Why this choice
api	FastAPI + Uvicorn	validation, claim-check in/out, tracking, UI, stats	OpenAPI docs for free, typed validation, streaming responses
worker-heavy	Celery (prefork)	PDFs and images (may need OCR), CPU-bound	isolated so OCR can never starve cheap jobs; memory-capped
worker-light	Celery (prefork)	txt/html/office + maintenance tasks	sub-second jobs keep low latency during OCR bursts
scheduler	Celery beat	recovery sweep every 20 s	periodic safety net that does not depend on any single worker
redis	Redis 7, AOF	message broker only	recommended stack; AOF keeps queued messages across restarts
postgres	PostgreSQL 16	documents, jobs, job_events	transactions, row locks (SKIP LOCKED), JSONB metadata, aggregates for the dashboard
minio	MinIO (S3 API)	raw documents and extraction results	claim-check store; same code works on AWS S3/GCS/R2

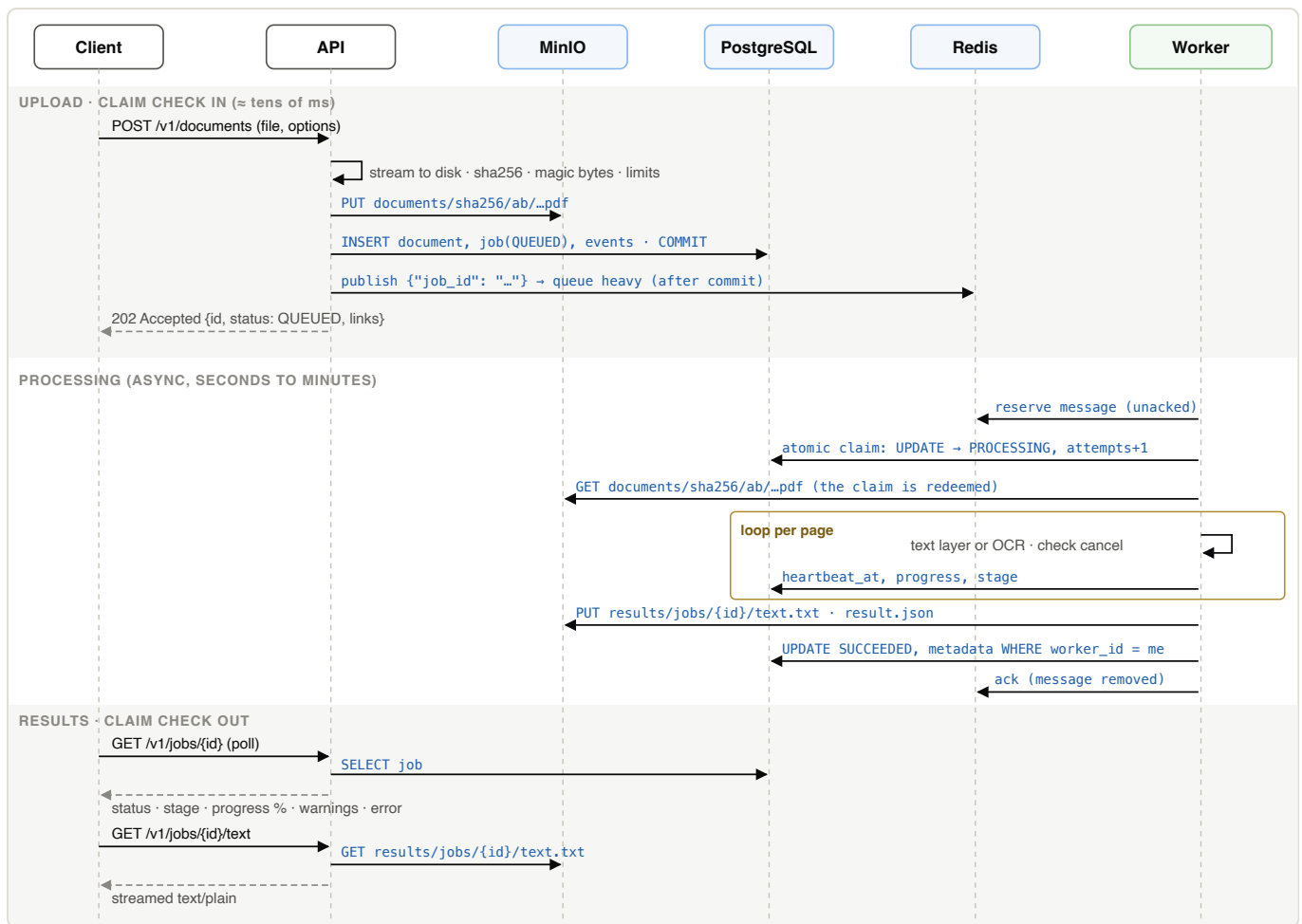


Figure 2. The claim-check lifecycle. The upload call returns as soon as the file is durable in object storage and the job row is committed. The message is published *after* the commit. Publishing first caused a real race during testing: a fast worker received the message before the row existed and dropped it.

3. Part 1 · Ingestion & storage

Claim-check implementation

1. **Stream and fingerprint.** The multipart body is streamed to a temp file in 1 MB chunks while a SHA-256 is computed. An outer ASGI middleware rejects oversized bodies *before* they are buffered, using `Content-Length` or by counting streamed bytes.
2. **Sniff the real type.** `libmagic` reads the magic bytes. ZIP containers are opened to tell DOCX, PPTX and XLSX apart, and text is checked with `charset-normalizer`. **Content wins over the extension:** a PNG named `.txt` is processed as an image, with a warning. Executables, archives and binary junk get **415** with the list of accepted extensions.
3. **Store the payload.** The object key is `documents/sha256/ab/<sha256>.ext`. Content addressing makes re-uploads free (HEAD first) and gives deduplication and caching at no extra cost.
4. **Record the claim check.** `documents` and `jobs` rows plus a `received` event are written in one transaction.
5. **Publish the ticket.** Only `{"job_id": ...}` is published, to the `heavy` or `light` queue depending on the kind. The response is `202 Accepted` with a `Location` header and links.

Messages stay around 1 KB whatever the document size. That keeps Redis memory flat and makes redelivery cheap. It also means the payload is stored once, in a system built for large objects.

Upload limits and their rationale

Limit	Value	Reasoning
Request size	50 MB	Covers roughly 300 pages of 150 dpi scans or any realistic office file, while bounding API disk and bandwidth. Checked before buffering, so it cannot fill the disk.
PDF pages	500	At $\approx 0.3\text{--}2$ s of OCR per page, 500 scanned pages already approach the 15-minute time limit.
Image pixels	80 MP	Guards against decompression bombs, since a 76 KB PNG can declare 400 MP (tested). PDF pages are rendered with a DPI capped to the same budget.
Office uncompressed size	300 MB	DOCX/PPTX/XLSX are ZIP files, so this is a zip-bomb guard. XLSX is also capped at 2 M cells.
Processing time	900 s soft / 960 s hard	The soft limit fails the job with <code>TIMEOUT</code> and an actionable message. The hard limit kills a stuck native call.
Attempts	3	Enough to ride out a crash or outage. A "poison" document that crashes workers stops after 3 tries.
Queue TTL	24 h	Guarantees a terminal state even when no worker ever comes back (<code>EXPIRED</code>).

4. Part 2 · Job tracking

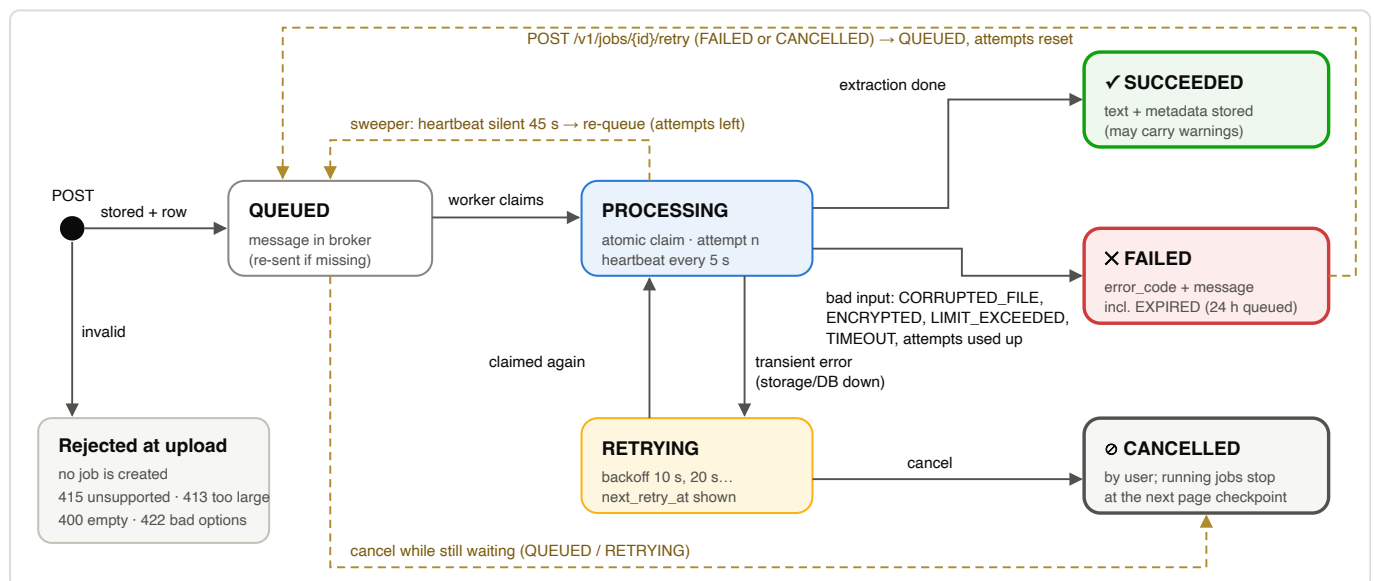


Figure 3. Job lifecycle. Solid arrows are worker transitions. Dashed arrows come from the recovery sweeper, the user or manual retries. Every path ends in one of three terminal states.

Data model (PostgreSQL)

Table	Key columns
documents	sha256, original_filename, kind, mime_type, size_bytes, storage_bucket/key

Tracking endpoints

Endpoint	Purpose
GET /v1/jobs/{id}	status + plain-language message, stage ("ocr: page 7/30"), progress, attempts, queue position, warnings, error, timings
GET /v1/jobs/{id}/events	timeline (received → dispatched → started → worker_lost → ...)

<code>jobs</code>	status, stage, stage_detail, progress, queue, options (JSONB), attempts/max_attempts, worker_id, heartbeat_at, next_retry_at, error_code/message, warnings (JSONB), result keys, text_preview, counts, metadata (JSONB), timestamps, cached_from_job_id, cancel_requested	GET <code>/v1/jobs</code>	list/filter by status, kind, filename, client_reference
		GET <code>/v1/jobs/{id}/result</code>	text + pages + metadata (409 with the reason if not ready)
		GET <code>/v1/jobs/{id}/text</code>	streamed plain text (download option)
<code>job_events</code>	append-only timeline: level, event, human message, data (JSONB)	POST <code>.../cancel</code> , <code>.../retry</code>	user control of the lifecycle

Document and job are separate because one file can be processed many times with different options, and the bytes are stored only once.

Why the database and not the Celery result backend? Celery's backend only knows `PENDING`. Its states are lost if Redis is flushed, and it cannot answer "how many OCR jobs failed today?". The job row is updated with a throttled progress write (at most 1 per second), a heartbeat every 5 s and ownership fencing (`WHERE worker_id = me`). Because of fencing, a worker the sweeper has already declared dead can never overwrite the newer attempt's result. `task_ignore_result=True` keeps Redis a pure transport.

5. Part 3 · Document intelligence

Every page must be accurate on scanned and photographed documents without wasting CPU on files that already contain text. The pipeline is therefore **hybrid**: it uses a native parser whenever the format carries text, and OCR only where it doesn't, decided *page by page*.

5.1 Extraction strategy per format

Kind	Method	Notes and safeguards
PDF	PyMuPDF <code>get_text("text", sort=True)</code> per page. A page is OCR'd when it has fewer than 25 non-blank characters, or when images cover more than 60% of it and it has under 200 characters. Such pages are rendered at 300 dpi in grayscale and passed to OCR.	In <code>auto</code> mode the richer result (native or OCR) wins. Rendering DPI is capped by the pixel budget. Damaged files are auto-repaired (with a warning), encrypted files are reported, and the page limit is enforced.
Images	EXIF orientation fix, 2× upscale if under 600 px, then OCR. Multi-page TIFF frames become pages.	Pillow <code>verify()</code> catches truncated files, and the decompression-bomb limit is 80 MP. When boxes are mostly vertical or confidence is low, Tesseract OSD detects 90/180/270° rotation and the page is OCR'd again.
Text family	BOM → UTF-8 → charset-normalizer. Newlines normalised; content with more than 5% control characters is rejected as binary.	Detected encoding and its confidence are returned as metadata. A warning is added below 0.9 confidence (e.g. Latin-1 files).
HTML	BeautifulSoup + lxml; <code>script/style/noscript/svg</code> removed	title, <code>lang</code> , meta description, link count
DOCX	python-docx, walking the body XML in order so tables stay in place (tab-separated), plus headers	zip integrity and uncompressed-size guard; core properties; warning if the document is mostly images
PPTX / XLSX	python-pptx: one page per slide, including grouped shapes, tables and speaker notes. openpyxl (read-only, cached formula values): one page per sheet.	zip guard; XLSX capped at 2 M cells with a truncation warning
RTF	striprtf	header check

Structured formats (DOCX/PPTX/XLSX/HTML/RTF) contain their text explicitly, so libraries were chosen for fidelity (reading order, tables, notes), permissive licences and maintenance rather than accuracy. Those libraries return the text exactly and the unit tests assert it. PDF and OCR have real accuracy and speed trade-offs, so those were benchmarked.

5.2 Benchmark

Method (`benchmark/` , reproducible with one `docker run`). There are 53 samples with known ground truth, in English and Spanish (accents, ñ, ¿, emails, amounts, dates). Twelve image conditions: clean, blur, heavy noise, 1–4° skew, 120 dpi, JPEG q15, perspective "photo", white-on-colour, two columns, 6 pt font, invoice and 2-page scanned PDFs. There are seven kinds of born-digital PDF: single column, two columns, table, 20 pages, out-of-order content stream, glyph-by-glyph positioning and `/Rotate 90` . The **assignment PDF** itself was added as a real document, rasterised at 150 and 300 dpi. The metrics are CER and WER (Levenshtein on normalised text), a reading-order-insensitive bag-of-words F1, warm s/page (mean and p95), cold start, peak RSS and install size. Each engine ran in its own process with the same 4 threads on arm64 (10 CPU, 7.8 GB).

OCR engine	Macro CER	Macro WER	BoW F1	s/page (p95)	Cold start	Peak RSS	Install	Licence
Tesseract 5.3 (LSTM)	0.214	0.230	0.826	0.28 (0.56)	0.08 s	252 MB	15 MB	Apache-2.0
Tesseract + OpenCV preprocessing	0.080	0.126	0.912	0.33 (0.73)	0.10 s	348 MB	88 MB	Apache-2.0
RapidOCR, PP-OCRv6 small (ONNX)	0.001	0.005	0.997	0.91 (1.64)	0.34 s	1.1 GB	221 MB	Apache-2.0
RapidOCR, PP-OCRv5 latin	0.002	0.006	0.997	0.94 (1.69)	0.34 s	1.1 GB	204 MB	Apache-2.0
EasyOCR 1.7 (PyTorch)	0.043	0.138	0.966	6.96 (13.0)	2.73 s	6.4 GB	812 MB	Apache-2.0

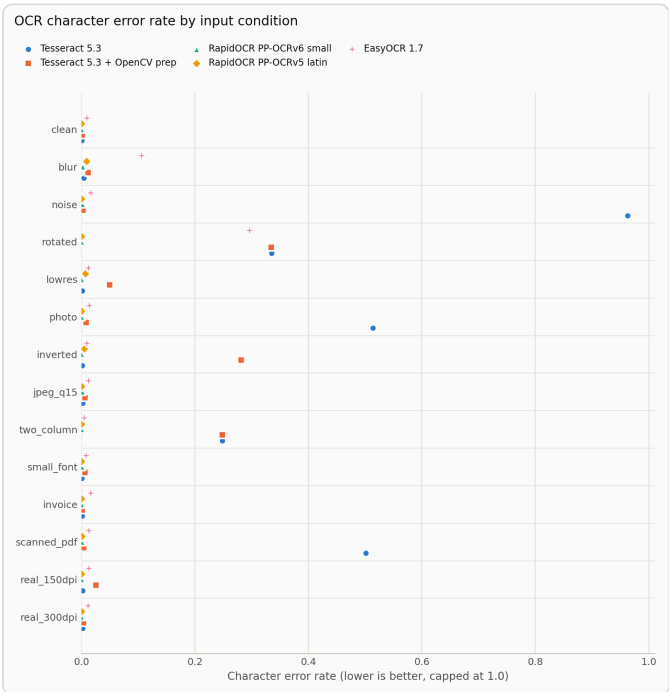


Figure 4. CER per condition. Raw Tesseract is excellent on clean input (0.001–0.002, including the real document) but collapses on noise (0.963, empty output), photos (0.514), noisy scanned pages (0.501), skew (0.335) and columns (0.248). OpenCV preprocessing fixes noise and photos but breaks white-on-colour text (0.281). RapidOCR stays at or below 0.009 in every condition.

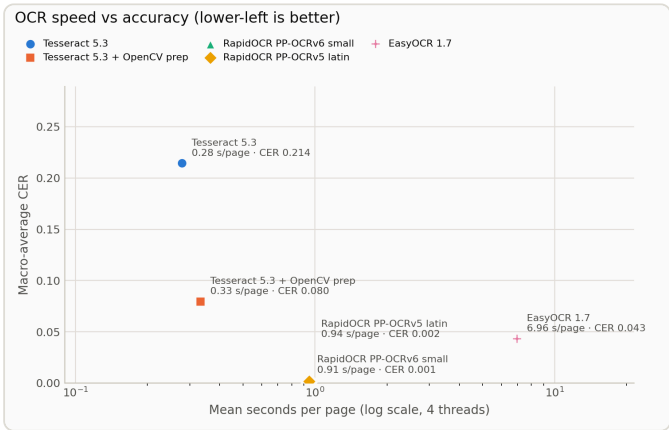


Figure 5. Speed vs accuracy (log x). EasyOCR is dominated: slower and less accurate than RapidOCR, and it needs 6.4 GB.

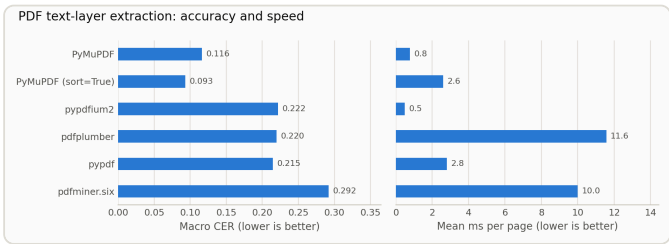


Figure 6. PDF text-layer extractors: macro CER and ms/page.

PDF extractor	Macro CER	ms/page	rotated page	glyph-positioned	out-of-order stream	table	Licence
PyMuPDF, sort=True	0.093	2.6	0.000	0.000	0.653	0.000	AGPL-3.0 / commercial
PyMuPDF (stream order)	0.116	0.8	0.000	0.000	0.813	0.000	AGPL-3.0 / commercial
pypdfium2	0.222	0.5	0.741	0.000	0.813	0.000	Apache-2.0 / BSD
pdfplumber	0.220	11.6	0.741	0.145	0.653	0.000	MIT
pypdf	0.215	2.8	0.000	0.690	0.813	0.000	BSD-3
pdfminer.six	0.292	10.0	1.249	0.000	0.147	0.648	MIT

All extractors were perfect on single-column, two-column and 20-page PDFs, and all returned **0 characters** for scanned pages. That validates the "few characters on this page, so OCR it" rule. No extractor recovers a content stream written out of visual order. That limitation is shared and is also why `ocr_mode=force` exists.

5.3 Decisions

PDF: PyMuPDF with `sort=True`

It was the only candidate without errors on rotated, glyph-positioned and table PDFs, at 2.6 ms/page. The same library also renders pages for OCR and repairs damaged files. **Caveat:** it is AGPL. For closed-source distribution, pypdfium2 (Apache/BSD, 0.5 ms/page) is the drop-in fallback, but rotated pages would need handling.

OCR default: RapidOCR (PP-OCRv6 small)

Its accuracy is two orders of magnitude better than raw Tesseract on degraded input, at ≈ 1 s/page on CPU. Models ship inside the wheel, are baked into the image and run fully offline. The line-orientation classifier is disabled because it flipped upright lines in the benchmark; page rotation is handled with Tesseract OSD instead.

Fast option: Tesseract (`ocr_engine=tesseract`)

About 3× faster with ¼ of the memory, and just as accurate on clean scans. It suits bulk, clean, born-scanned archives. It is kept installed anyway for OSD and language validation.

Rejected: EasyOCR, VLM/layout pipelines

EasyOCR is dominated (7× slower, less accurate, 6.4 GB). Docling, Marker and Nougat were excluded because of GPU/RAM needs on the CPU-only target, and RapidOCR already reaches near-zero CER on the test set.

Measured in the running service (30-page scanned PDF rendered at 300 dpi, worker-heavy with 2 processes × 2 threads): RapidOCR took **51.8 s** (1.7 s/page including rendering and orientation checks). Tesseract took **27.1 s**. Peak memory of the heavy worker container was 1.3 GB with both engines running at once. On a photographed invoice, RapidOCR occasionally merged words ("ACMEData") and read table cells as separate lines, while Tesseract kept table rows together on clean input. That is why both engines are exposed.

5.4 Where and how the extracted text is stored

Artifact	Location	Why there
Full text (pages separated by form feed \f)	MinIO <code>results/jobs/{id}/text.txt</code>	Can be megabytes. Streamed straight to clients, cheap storage with lifecycle rules, and keeps the database small and fast.
Structured result: per-page text, method (<code>text_layer / ocr:rapidocr</code>), OCR confidence, duration, warnings, metadata	MinIO <code>results/jobs/{id}/result.json</code>	One self-contained, versionable document for downstream pipelines (search indexing, embeddings, RAG).

Queryable summary: text preview (1 000 chars), char/word/page counts, language, metadata JSONB, warnings	PostgreSQL <code>jobs</code>	Powers listings, filters and the dashboard without touching object storage.
--	------------------------------	---

The raw document stays content-addressed in `documents/`, so any job can be reprocessed with new options (`force=true` or `retry`) without re-uploading.

5.5 Metadata worth extracting

Group	Fields	Use
Always	SHA-256, detected MIME and kind, size, page count, char/word/line counts, reading time, extraction method per page, mean OCR confidence, timings	dedup, cost accounting, quality triage
Language	ISO code + confidence (fastText lid.176, on-device, ~1 MB)	route to language-specific NLP, choose the OCR language
Entities (regex)	emails, URLs, dates (ISO / numeric / Spanish and English month names), amounts (MXN/USD/EUR/\$), phone numbers	invoice and form triage without an LLM
PDF	title, author, subject, keywords, creator/producer, created/modified, encryption, repaired flag, page size, outline presence, text-layer vs OCR pages	provenance, detecting scanned vs digital
Image	format, dimensions, DPI, frames, EXIF (camera make/model, date, software, orientation)	quality warnings (low resolution), provenance
Office / HTML / text	core properties (title, author, dates), paragraph/table/image counts, slide and sheet names, HTML title/lang/description, text encoding	search facets, display

6. Part 4 · Queue design (Redis + Celery)

Setting	Value	Effect / reason
Queues	<code>heavy</code> , <code>light</code> , <code>maintenance</code>	Routing is decided at upload from the detected kind. A burst of 500-page scans cannot delay a 1 KB text file, and each pool can be scaled on its own (<code>--scale worker-heavy=3</code>).
<code>task_acks_late</code>	True	The message is acknowledged only after the task finishes, so a crash leaves it unacked and it gets redelivered.
<code>task_reject_on_worker_lost</code>	True	If a pool child is OOM-killed, the message is queued instead of being marked failed.
<code>worker_prefetch_multiplier</code>	1	A busy worker doesn't reserve messages it can't start, so another replica can take them.
<code>visibility_timeout</code>	hard limit + 300 s	Redis redelivers unacked messages only after the longest legitimate task has had time to finish.
Concurrency	heavy 2 processes × OCR_THREADS=2 (4 GB limit), light 4	Threads per process are pinned (<code>OMP_THREAD_LIMIT</code> , <code>ONNX</code> intra-op), so processes × threads ≈ cores reserved for OCR with no oversubscription. RapidOCR peaks at ~1.1 GB per process.
<code>worker_max_tasks_per_child</code>	200	Recycles processes to contain leaks in native libraries (MuPDF, Leptonica).
Time limits	soft 900 s / hard 960 s	A runaway job becomes a clear <code>TIMEOUT</code> failure instead of a stuck slot.

Task id	<code><job_id>. <dispatch_n></code>	Lets the sweeper map broker messages back to jobs, and keeps each re-dispatch distinguishable in Flower.
Redis persistence	AOF, <code>everysec</code>	Queued messages survive a broker restart, losing at most 1 s of messages. The sweeper covers even that.

At-least-once delivery plus an idempotent claim gives effectively-once processing. A worker starts a job only through an atomic `UPDATE jobs SET status='PROCESSING', attempts=attempts+1 ... WHERE id=? AND status IN ('QUEUED','RETRYING') AND attempts < max_attempts RETURNING id`. Duplicate deliveries, redelivered messages and re-dispatches find the row not claimable and are acknowledged in about 3 ms without doing any work.

7. Part 5 · Failure handling

Failure matrix

Failure	Mechanism	What the user sees
Unsupported, empty or oversized upload	Magic-byte sniffing, size guard before buffering	415 / 400 / 413 with <code>error.code</code> , message and accepted extensions. No job is created.
Corrupted / truncated / encrypted / bomb file	Typed extractor errors (<code>retryable=False</code>), so there is no pointless retry. MuPDF repairs damaged PDFs when it can.	<code>FAILED + CORRUPTED_FILE</code> , <code>ENCRYPTED_DOCUMENT</code> or <code>LIMIT_EXCEEDED</code> , or <code>SUCCEEDED</code> with a "PDF was repaired" warning.
Worker process or container killed mid-job	Heartbeat stops. The sweeper finds <code>PROCESSING</code> with a heartbeat older than 45 s, re-queues and re-dispatches. The message is also unacked (<code>acks_late</code>).	Timeline shows <code>worker_lost</code> and attempt 2/3. A heartbeat-age warning appears while waiting.
Poison document (crashes every time)	Attempts are counted in the claim; after 3 the sweeper fails the job.	<code>FAILED WORKER_LOST</code> : "stopped responding on 3 of 3 attempts... makes the worker crash or run out of memory".
Whole stack killed	Durable volumes. On start, every worker runs a sweep (advisory-locked) and beat continues every 20 s.	Jobs resume by themselves: running ones on attempt 2, queued ones from the AOF-restored queue.
Broker loses messages / publish fails	The sweeper compares <code>QUEUED</code> jobs against the Redis lists and <code>unacked</code> hash and re-dispatches missing ones. Duplicates are harmless.	Upload still returns 202 with a "queue temporarily unavailable" warning. Events show <code>dispatch_failed</code> / <code>message_missing</code> .
Object storage / DB outage	API maps errors to 503 + <code>Retry-After</code> . Workers classify them as transient: <code>RETRYING</code> with exponential backoff (10 s, 20 s...).	<code>STORAGE_UNAVAILABLE</code> / <code>DATABASE_UNAVAILABLE</code> . The job shows <code>next_retry_at</code> and the attempt counter.
Slow or stuck processing	Soft/hard time limits; cancellation checked per page	<code>FAILED TIMEOUT</code> with advice, or <code>CANCELLED</code> .
No workers at all	Queue TTL expiry; readiness endpoint	<code>/v1/system</code> : "No workers online: jobs will wait...". After 24 h: <code>FAILED EXPIRED</code> .
Low-quality result	Per-page confidence, text-layer detection, encoding confidence	Warnings: low OCR confidence on pages 3–5, mixed document, no text found, extension mismatch, rotation corrected...

Chaos test results

`scripts/chaos_test.py` injects each failure into the running compose stack and asserts the outcome from the public API only.

Scenario	Result	Expected	Observed (from the public API)	Time
rejected_inputs	✓ PASS	unsupported/empty → 4xx at upload; damaged files → terminal state with explicit error code (repairable PDFs succeed with a warning)	program.exe → HTTP 415; archive.zip → HTTP 415; empty.txt → HTTP 400; binary_named_as.txt → HTTP 415; corrupted.pdf → SUCCEEDED (warnings: 3); garbage.pdf → FAILED CORRUPTED_FILE; truncated.jpg → FAILED CORRUPTED_FILE; encrypted.pdf → FAILED ENCRYPTED_DOCUMENT; huge_400mp.png → FAILED LIMIT_EXCEEDED	6s
worker_kill	✓ PASS	job re-queued after heartbeat timeout and SUCCEEDED on attempt 2	SUCCEEDED after 2 attempts; recovered+finished 98s after kill; events: received → dispatched → started → worker_lost → dispatched → started → succeeded	108s
cancel_running	✓ PASS	a running job stops at the next page and ends CANCELLED	CANCELLED at 11%	6s
broker_data_loss	✓ PASS	sweeper detects jobs missing from the broker and re-dispatches them	['SUCCEEDED', 'SUCCEEDED', 'SUCCEEDED'] in 61s; message_missing events: True	64s
broker_down_on_upload	✓ PASS	upload still accepted (202); job dispatched automatically when the broker returns	HTTP 202; final SUCCEEDED; events: received → dispatched → dispatch_failed → message_missing → dispatched → started → succeeded	19s
storage_down_during_processing	✓ PASS	job goes RETRYING with backoff, then SUCCEEDED; uploads during the outage get 503 STORAGE_UNAVAILABLE	upload during outage → HTTP 503 STORAGE_UNAVAILABLE; job SUCCEEDED after 2 attempts; events: received → dispatched → started → retry_scheduled → dispatched → started → succeeded	17s
database_down	✓ PASS	HTTP 503 DATABASE_UNAVAILABLE while down, /v1/system reports degraded, recovers automatically	/v1/jobs → 503 DATABASE_UNAVAILABLE; /v1/system → 503 ['Database unavailable: uploads and status queries will fail until it recovers.']; after restart → 200	11s

stack_kill	✓ PASS	every job (queued, running, finished) reaches SUCCEEDED after a full restart	scanned_report_30p.pdf: PROCESSING→SUCCEEDED (attempts 2); scanned_letter.pdf: PROCESSING→SUCCEEDED (attempts 2); mixed_text_and_scan.pdf: QUEUED→SUCCEEDED (attempts 1); report.docx: SUCCEEDED→SUCCEEDED (attempts 1); invoice_photo.jpg: QUEUED→SUCCEEDED (attempts 1)	106s
------------	--------	--	---	------

Run (2026-09-16 21:14) on the compose stack (arm64, OrbStack). Worker loss is detected after 45 s without a heartbeat; the sweeper runs every 20 s. Recovery time therefore includes up to about 65 s of detection plus the re-run.

8. Part 6 · Monitoring & UI

Monitoring works on three levels. Each one answers a different question.

Tool	Question it answers	Contents
Dashboard /dashboard	"Is the pipeline healthy and keeping up?" (product and operations view)	dependency health pills; tiles for received, success rate, in-flight, p50/p95 processing time, queue wait, pages, jobs with warnings; throughput chart (succeeded vs failed per interval); queue depths and per-worker busy slots; jobs and average time by document type; failures by error code; reliability events (worker losses recovered, lost messages re-dispatched, automatic retries, uploads while the broker was down); active jobs with heartbeat age; recent failures. Window 15 min–7 days, auto-refresh 3 s.
Flower :5555	"What is Celery doing right now?" (task level)	per-task state, runtime and args (the job id), worker pools, broker queues, rate limits and remote control
GET /v1/system	"Can I use the service?" (for machines and the UI banner)	DB, object storage, broker and worker status with human messages; HTTP 503 when degraded

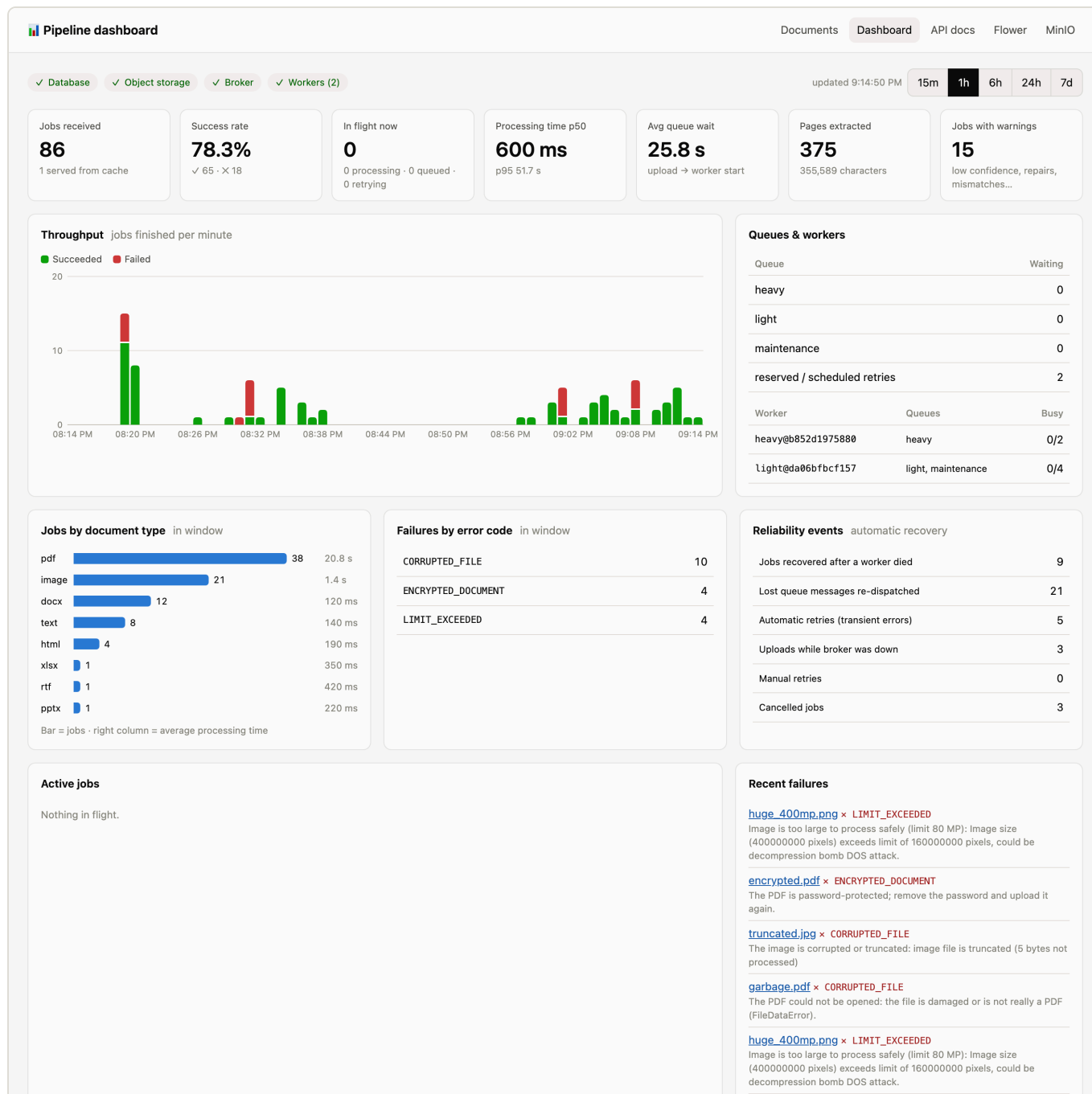


Figure 7. Monitoring dashboard after the chaos test run. The reliability panel counts the automatic recoveries, and the failure panel shows the explicit error codes of the malformed inputs.

Document Intelligence

All systems operational

DocumentsDashboardAPI docsFlower

Upload documentsmax 50 MB · 500 PDF pages

Drop files here or click to choose

pdf · png · jpg · jpeg · tif · tiff · webp · bmp · gif · txt · md · markdown · csv · tsv · log · json · xml · html · htm · xhtml · docx · pptx · xlsx · rtf

OCR languages

eng+spa

OCR mode

auto (only pages with

OCR engine

rapidocr

Client reference

optional

☒ Extract entities

☐ Re-process duplicates

Jobs

Filter by filename

All statuses

Document	Status	Progress	Pages	Time	Created
invoice_photo.jpg image · 126.9 KB	SUCCEEDED	done	1	1.5 s	9:12:37 PM
report.docx docx · 36.4 KB	SUCCEEDED	done	1	30 ms	9:12:37 PM
mixed_text_and_scan.pdf pdf · 206.7 KB · 1	SUCCEEDED	done	2	1.7 s	9:12:37 PM
scanned_letter.pdf pdf · 618.7 KB	SUCCEEDED attempt 2/3	done	3	4.2 s	9:12:37 PM
scanned_report_30p.pdf pdf · 4.4 MB	SUCCEEDED attempt 2/3	done	30	50.9 s	9:12:37 PM
report.docx docx · 36.4 KB	SUCCEEDED attempt 2/3	done	1	100 ms	9:12:12 PM
readme.md text · 685 B	SUCCEEDED	done	1	60 ms	9:11:51 PM
article.html html · 983 B	SUCCEEDED	done	1	60 ms	9:10:49 PM
invoice_scan.png image · 69.0 KB	SUCCEEDED	done	1	1.3 s	9:10:49 PM
report.docx docx · 36.4 KB	SUCCEEDED	done	1	150 ms	9:10:49 PM
scanned_report_30p.pdf pdf · 4.4 MB	CANCELLED	done	—	5.2 s	9:10:41 PM
scanned_report_30p.pdf pdf · 4.4 MB	SUCCEEDED attempt 2/3	done	30	50.5 s	9:08:53 PM

mixed_text_and_scan.pdf

SUCCEEDED

Text extracted successfully.

1 warning

Mixed document: pages 2 had no usable text layer and were OCR'd.

↓.txt

{ }JSON

Original

Job

3388be87-fffc-479d-a447-70c62635b8a0

Type

pdf · application/pdf · 206.7 KB

Options

language=eng+spa

ocr_mode=auto

ocr_engine=rapidocr

extract_entities=true

Queue

heavy · attempt 1/3

Timing

waited 15.8 s · processed in 1.7 s

Result

2 page(s) · 196 words · 1,258 chars · language es (73%) · OCR confidence 99.3

Document metadata

Timeline (4)

Extracted text

All12-ocr

Real services rarely process a request synchronously, and file ingestion especially. Documents arrive faster than they can be processed, so a reliable solution accepts the upload, stores it, queues the work and lets users track progress asynchronously.

The claim-check pattern keeps messages small: the heavy payload is stored in object storage and only a reference travels through the queue. Workers redeem the claim check when they are ready to do the work.

Failure handling matters as much as the happy path. Workers can be killed in the middle of a task, brokers can restart and files can be corrupted. No job may be lost and no document may stay pending forever.

La digitalización de documentos permite que la información sea buscable y reutilizable. ¿Cuántas páginas escaneadas existen todavía en los archivos de una universidad? Probablemente millones. El reconocimiento óptico de caracteres convierte imágenes en texto. La precisión depende de la resolución, del contraste y de la tipografía; por eso conviene medir antes de elegir una herramienta.

Figure 8. Upload UI: drag and drop with options, a live job list (status badge, progress, stage) and job detail with warnings, entities, document metadata, the event timeline and per-page text (OCR pages marked with confidence).

 Flower Workers Tasks Broker Documentation 					
Show 15 tasks		Search:			
Name	UUID	State	args	kwargs	Result
docintel.recover_jobs	26a2a3f7-fb7d-4673-ba1a-1f17573039ab	SUCCESS	()	{}	{'requeued': 0, 'failed': 0}
docintel.recover_jobs	dfa1d1f6-a5f5-4ec5-8e3e-3ab8e91b11c3	SUCCESS	()	{}	{'requeued': 0, 'failed': 0}
docintel.recover_jobs	6f2f1a77-2ace-4531-9f1b-5b60ad153402	SUCCESS	()	{}	{'requeued': 0, 'failed': 0}
docintel.process_document	359f15dd-782c-4ae9-8dc4-60c18c24c0f0.2	SUCCESS	()	{'job_id': '359f15dd-782c-4ae9-8dc4-60c18c24c0f0'}	{'job_id': '359f15dd-782c-4ae9-8dc4-60c18c24c0f0'}
docintel.process_document	e93ec55d-a2e8-48cd-8ea5-f115df97d32b.2	SUCCESS	()	{'job_id': 'e93ec55d-a2e8-48cd-8ea5-f115df97d32b'}	{'job_id': 'e93ec55d-a2e8-48cd-8ea5-f115df97d32b'}
docintel.recover_jobs	181f71f2-48f0-4065-ae42-469c0c98e00d	SUCCESS	()	{}	{'requeued': 2, 'failed': 0}
docintel.recover_jobs	7b7891ce-e739-4123-9b14-1de582988426	SUCCESS	()	{}	{'requeued': 0, 'failed': 0}
docintel.recover_jobs	6a79e6b8-8073-4b1d-9cb9-783514a1d211	SUCCESS	()	{}	{'requeued': 0, 'failed': 0}
docintel.process_document	405c9f19-8893-4543-8c87-34d323f9721d.1	SUCCESS	()	{'job_id': '405c9f19-8893-4543-8c87-34d323f9721d'}	{'job_id': '405c9f19-8893-4543-8c87-34d323f9721d'}
docintel.process_document	3388be87-fffc-479d-a447-70c62635b8a0.1	SUCCESS	()	{'job_id': '3388be87-fffc-479d-a447-70c62635b8a0'}	{'job_id': '3388be87-fffc-479d-a447-70c62635b8a0'}
docintel.process_document	58ab94c5-0194-443a-8d7b-28d414bf189f.1	SUCCESS	()	{'job_id': '58ab94c5-0194-443a-8d7b-28d414bf189f'}	{'job_id': '58ab94c5-0194-443a-8d7b-28d414bf189f'}
docintel.process_document	359f15dd-782c-4ae9-8dc4-60c18c24c0f0.1	STARTED	()	{'job_id': '359f15dd-782c-4ae9-8dc4-60c18c24c0f0'}	
docintel.process_document	e93ec55d-a2e8-48cd-8ea5-f115df97d32b.1	STARTED	()	{'job_id': 'e93ec55d-a2e8-48cd-8ea5-f115df97d32b'}	
docintel.recover_jobs	724d42d0-d07d-412e-9bf2-6786365394cc	FAILURE	()	{}	
docintel.process_document	989ab19b-3b5c-47f7-9d39-e4d1be3cd436.2	SUCCESS	()	{'job_id': '989ab19b-3b5c-47f7-9d39-e4d1be3cd436'}	{'job_id': '989ab19b-3b5c-47f7-9d39-e4d1be3cd436'}

Figure 9. Celery Flower. Task ids are `<job_id>.<dispatch_n>`, so re-dispatches after a recovery are visible as `.2`, `.3`.

9. Trade-offs, limits & future work

Decision	Cost we accept	Mitigation / next step
Redis as broker (not RabbitMQ/SQS)	Weaker delivery guarantees: unacked messages come back only after the visibility timeout, and FLUSHALL loses everything.	Postgres is the source of truth and the sweeper re-dispatches from it (tested). Swap to RabbitMQ quorum queues if the message volume grows.
Heartbeat-based loss detection	Recovery latency is 45–65 s. A 60 s network partition could cause a redundant re-run.	Ownership fencing makes a late worker's writes no-ops. Both timeouts are env-tunable.
Job-level (not page-level) retries	A crash at page 29/30 re-OCRs the whole document.	Split large PDFs into page-range subtasks (Celery chord) with per-page checkpoints in object storage.
Polling API	Clients must poll.	Webhook (<code>callback_url</code>) exists. Server-Sent Events on <code>/v1/jobs/{id}/stream</code> would be the next step.
PyMuPDF (AGPL)	Licence obligations if the service is distributed closed-source.	pypdfium2 fallback (benchmarked) or a commercial MuPDF licence.
RapidOCR default	1.1 GB peak per process, $\approx 3\times$ slower than Tesseract, and it sometimes drops spaces on photos or splits table rows.	<code>ocr_engine=tesseract</code> for clean bulk scans; worker memory limit and concurrency sized for it.
Regex entities, flat text	No table structure, key-value pairs or handwriting.	Keep per-page boxes. Add layout/table models (e.g. Docling) as an optional <code>layout</code> queue on GPU nodes.

Out of scope for a class project, and required before production: authentication with API keys or OAuth, per-tenant quotas and rate limiting, an allow-list for `callback_url` (SSRF risk), antivirus scanning (ClamAV) before processing, encryption at rest and retention policies (S3 lifecycle rules plus deleting DB rows), Alembic

migrations instead of `create_all`, Prometheus metrics with alerting on queue age, and autoscaling workers on queue depth (e.g. KEDA).

10. Appendix · How to run & reproduce

```
docker compose up --build          # api :8000 (UI, /dashboard, /docs) · flower :5555 · minio :9001
docker compose run --rm --no-deps api pytest -q      # 32 unit tests
python3 scripts/chaos_test.py          # failure injection → docs/chaos_results.md
docker build -t docintel-bench benchmark/
docker run --rm -v "$PWD/benchmark":/bench docintel-bench # benchmark
```

Path	Content
app/api	FastAPI app, schemas, dashboard aggregations
app/worker	Celery config, processing task, heartbeat, progress, retries, webhooks
app/jobs.py	dispatch-after-commit, atomic claim, recovery sweeper
app/extraction	format detection, PDF/image/text/office extractors, OCR engines, enrichment
benchmark/	dataset generator, runner, raw and summarised results, charts
samples/	demo documents and 11 edge cases